

Team Report: Project Reflection

1. Roles & Responsibilities

This project required a clear division of labor to manage the complexity of network programming. Each member focused on a core module while ensuring compatibility with the rest of the system:

- **ADAM ZAKIE – Server Architecture & Concurrency:** Managed the development of ChatServer.java and ClientHandler.java. This role was responsible for ensuring the server could handle multiple simultaneous client connections using multithreading and correctly broadcasting messages to all active participants.
- **AQIL – Client Interface & Networking:** Developed ChatUI.java and handled the socket lifecycle. This involved designing a user-friendly Java Swing interface and implementing the client-side logic to handle server connection, authentication, and the background thread required to listen for incoming messages without freezing the GUI.
- **HASHRY – Security Integration & Data Encryption:** Spearheaded the security implementation via EncryptionUtils.java using AES encryption to ensure end-to-end data protection. This role ensured that the message payload was consistently encrypted before transmission and decrypted upon receipt across all modules.
- **SHADZARIL – Quality Assurance & Project Documentation:** Managed the creation of the User Manual and Technical Maintenance Documentation. This role acted as the "user," performing rigorous stress testing to identify bugs, verifying that the authentication handshake failed gracefully when incorrect passwords were used, and ensuring all project deliverables were clear and well-formatted for final submission.

2. Challenges Faced & Solutions

Challenge	Technical Context	Solution
GUI Lifecycle Management	Encountered the "Class does not have a main method" error when attempting to run the JFrame directly.	We researched the Java Swing threading model and implemented a public static void main method utilizing <code>SwingUtilities.invokeLater</code> to ensure the GUI is created on the Event Dispatch Thread (EDT).
Git Authorization	Received "not authorized" errors when pushing to GitHub from the NetBeans IDE.	Diagnosed that GitHub deprecated account password authentication for command-line tools. We generated a Personal Access Token (PAT) with repository scope and updated the local Git credential manager to authenticate securely.
Concurrency / Data Race	Observed synchronization issues where multiple clients writing to the same JTextArea caused visual lag.	Implemented a dedicated thread for the <code>BufferedReader</code> listener in the <code>ChatUI</code> class, ensuring that the GUI update logic remains separate from the network I/O loop.

3. Lessons Learned

- **Separation of Concerns:** We learned that tightly coupling network code with UI logic is a major pitfall. By separating `EncryptionUtils` and `ClientHandler` from the `ChatUI`, we made the code significantly easier to debug and maintain.
- **Multithreading:** We gained a deeper understanding of Java's concurrency. We realized that for a chat application to be usable, the network listener *must* run on its own thread, otherwise, the application would hang while waiting for data.
- **Toolchain Proficiency:** Beyond just writing code, we learned how to effectively use industry-standard tools like GitHub and NetBeans debugging. Troubleshooting the Git authentication taught us the importance of modern security practices like token-based authentication.